



CURSIS PLATFORM

FULL ARCHITECTURE & ENGINEERING HANDBOOK

A comprehensive deep-dive into the Autonomous Workspace Orchestration Platform, its high-performance Next.js 16 stack, Three.js spatial engine, Ordis AI orchestration pipeline, and step-by-step build blueprint.

NEXT.JS 16 (APP ROUTER)

REACT 19

TYPESCRIPT 5

THREE.JS + R3F

FIREBASE ADMIN + FIRESTORE

ORDIS AI ENGINE

VERCEL EDGE

DOCUMENT TYPE

Architecture & Technical Manual

TARGET ARCHITECTURE

Full-Stack Serverless Edge + AI

REPOSITORY

github.com/mukulk6776/Cursis

LIVE PRODUCTION URL

<https://cursis.vercel.app>

EXECUTIVE OVERVIEW

Cursis reimagines workspace productivity through autonomous ambient intelligence. Moving beyond passive Kanban boards and static CRMs, Cursis acts as an autonomous operating partner (Ordis Engine) that ingests meeting dialogues, detects project risk bottlenecks, orchestrates CRM workflows, and safely executes multi-step actions with human-in-the-loop validation.

1. EXECUTIVE SUMMARY & CORE PHILOSOPHY

What is Cursis?

Cursis is a modern, high-contrast, AI-orchestrated workspace operating system tailored for high-velocity software agencies, product teams, and founders. Unlike traditional SaaS tools that require manual data entry, Cursis combines ambient intelligence with high-performance real-time interfaces.

1. Autonomous AI Intelligence (Ordis)

Continuously scans workspaces, parses raw meeting transcripts into actionable tickets, estimates timelines, and surfaces proactive approval recommendations before deadlines slip.

2. Somba Neo-Brutalist Visual System

Striking visual identity featuring high-contrast stark borders, sharp geometry, rich typography (Space Grotesk + JetBrains Mono), smooth dark modes, and micro-animations.

3. Immersive 3D Spatial Canvas

Hardware-accelerated WebGL spatial layers powered by Three.js and React Three Fiber to visualize complex multi-agent workflows and interactive project topologies.

4. Enterprise-Grade Serverless Engine

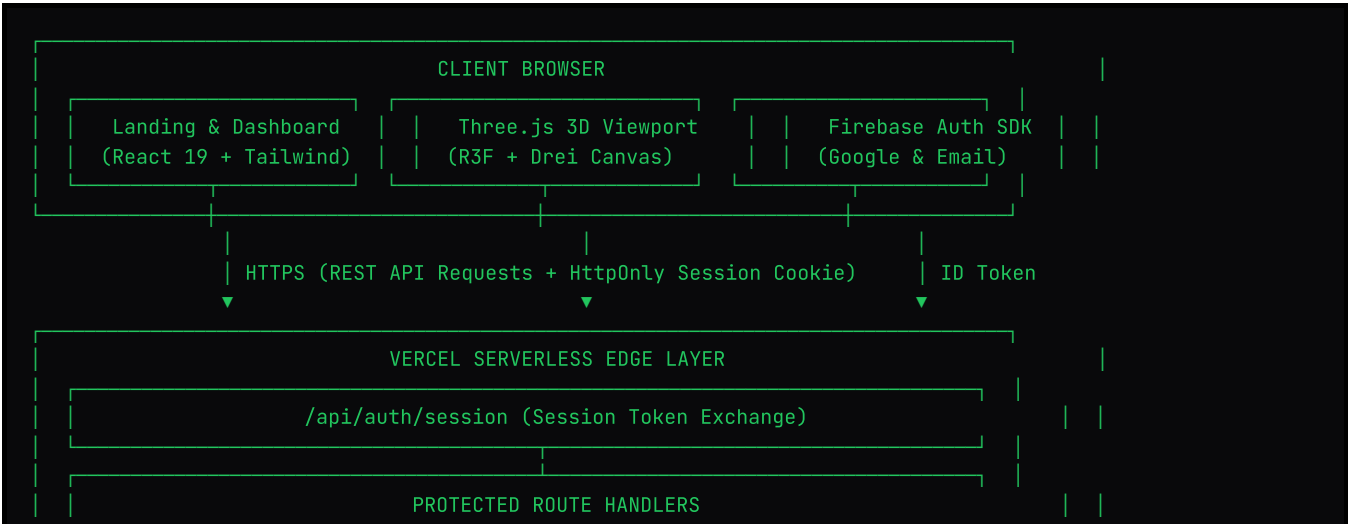
40+ dedicated serverless API route handlers deployed across global edge networks with resilient tamper-proof session token authorization and Cloud Firestore storage.

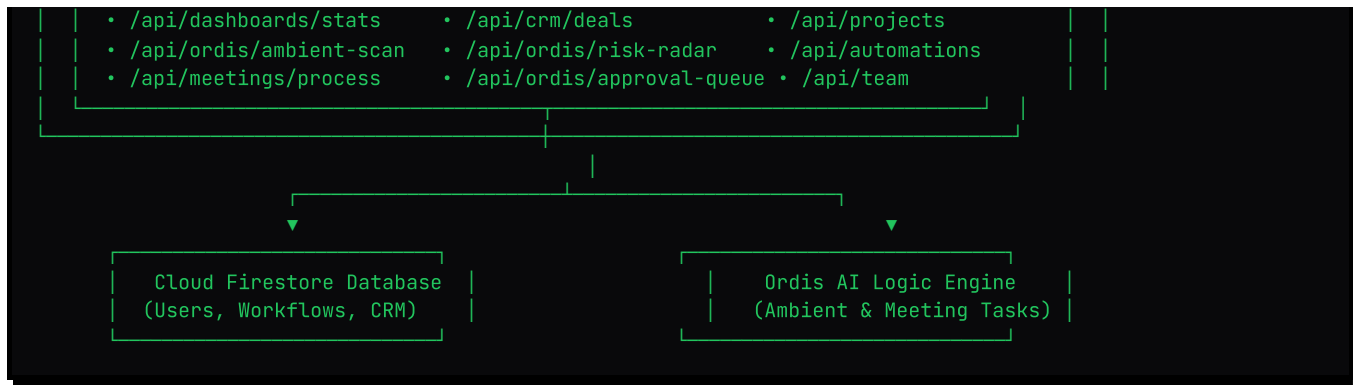
2. COMPREHENSIVE TECHNOLOGY STACK

Every component in the Cursis ecosystem is selected for maximum speed, strict type safety, zero layout shift, and fluid user experience.

LAYER / DOMAIN	TECHNOLOGY / PACKAGE	ROLE & STRATEGIC JUSTIFICATION
Core Framework	<code>Next.js 16.3.2</code> (App Router)	Hybrid Server Components (RSC) and Dynamic Edge Route Handlers with Turbopack compilation.
UI Runtime	<code>React 19.2.8</code> + <code>TypeScript 5</code>	Latest concurrent rendering features, Actions, and strict end-to-end type safety.
Styling & CSS	<code>Tailwind CSS v4</code> + PostCSS	Next-generation CSS engine providing zero-runtime styling with custom Somba Neo-Brutalist tokens.
Spatial & 3D	<code>Three.js</code> + <code>@react-three/fiber</code> + <code>@react-three/drei</code>	Interactive 3D geometry, spatial lighting, and dynamic visual nodes on the canvas.
Motion & Scroll	<code>Framer Motion v13</code> + <code>Lenis 1.3</code>	Physics-based spring transitions, layout morphing, and smooth inertia scroll momentum.
Icons & Theming	<code>Lucide React</code> + <code>next-themes</code>	Clean SVG vector iconography with persistent dark/light theme switching.
Client Auth	<code>Firebase v12.18.0</code> (Auth)	Client-side Google OAuth popup and email/password identity management.
Server Backend	<code>Firebase Admin v14.3.0</code> + <code>Node.js</code>	Server-side Firestore document persistence, custom claims, and secure session management.
Security / Session	<code>HttpOnly</code> Encrypted Session Cookies	Self-contained, cross-domain cookie exchange (<code>cursis_session</code>) with 5-day expiration.
Hosting & CI/CD	<code>Vercel Serverless Platform</code>	Automated Git-triggered builds with global edge distribution and zero-config caching.

High-Level Architecture Topology





3. DETAILED PLATFORM MODULES & FEATURE ENGINE

1. Ordis AI Autonomous Workspace Partner

The cornerstone of Cursis is **Ordis**—an autonomous multi-agent engine designed to eliminate manual management overhead. Key subsystems include:

- **Ambient Workspace Scanner** (`/api/ordis/ambient-scan`): Continuously audits active tickets, unassigned tasks, and upcoming milestone dates to generate automated health scores.
- **Meeting Intelligence Processor** (`/api/ordis/process-meeting`): Converts raw transcripts into structured action items, assigns owners, updates deal stages, and auto-schedules calendar follow-ups.
- **Predictive Risk Radar** (`/api/ordis/risk-radar`): Simulates project delivery timelines based on team capacity and flags overdue risks before they impact client deliverables.
- **Autonomous Approval Queue** (`/api/ordis/approval-queue`): Holds high-impact AI actions (sending emails, modifying client contracts, reallocating budgets) in a safety queue for 1-click human verification.

2. Executive Dashboard

Central command displaying live MRR, active sprint velocity, workload distribution charts, Ordis AI saved hours, and priority action feeds.

3. High-Velocity CRM & Deals

Complete pipeline management supporting drag-and-drop deal progression, customer contact tracking, lead scoring, and automated renewal triggers.

4. Project & Task Engine

Unified task management with multi-view modes (Kanban, List, Timeline), automated workload rebalancing, and smart dependency tracking.

5. Agency Catalog & Custom Tools

Modular tool marketplace allowing workspaces to dynamically request, build, and deploy custom internal tools via the Ordis builder API.

4. DATA SCHEMA & SESSION SECURITY ARCHITECTURE

Cursis uses a resilient multi-layered data model backed by Cloud Firestore and self-contained encrypted session cookies for reliable serverless execution.

COLLECTION	KEY DOCUMENT FIELDS	DESCRIPTION & BUSINESS LOGIC
users	uid, email, displayName, role, photoURL, createdAt	Stores user profile records and workspace permissions (Owner, Admin, Member).
workspaces	id, name, ownerId, plan, members, settings	Multi-tenant workspace container isolating data between distinct teams.
workflows	id, userId, prompt, name, status, runs, createdAt	User-created and AI-orchestrated automation pipelines.
deals	id, title, value, stage, contactEmail, riskLevel	CRM sales pipeline opportunities tracked against revenue targets.
tasks	id, title, assignee, status, priority, dueDate	Actionable project work items tied to sprints and milestones.
audit_logs	id, actor, action, timestamp, metadata, rollbackToken	Immutable security audit trail enabling 1-click rollback of AI and user actions.

5. STEP-BY-STEP GUIDE: HOW TO BUILD CURSIS FROM SCRATCH

Follow this exact chronological blueprint to recreate or extend the Cursis platform.

Step 1: System Prerequisites & Workspace Initialization

Ensure Node.js 20+ (LTS) and Git are installed. Clone the repository and install all dependencies:

```
# 1. Clone the repository
git clone https://github.com/mukul6776/Cursis.git
cd Cursis

# 2. Install dependencies (Next.js 16, React 19, Three.js, Firebase)
npm install
```

Step 2: Firebase Console & Firestore Setup

1. Navigate to <https://console.firebase.google.com/> and create a new project (e.g. `cursis-prod`).
2. Under **Authentication** > **Sign-in method**, enable **Email/Password** and **Google Sign-In**.
3. Under **Firestore Database**, click **Create database** in Production mode.
4. Under **Project Settings** > **General** > **Your apps**, register a Web App and copy the client config values (API Key, Project ID, App ID, etc.).
5. Under **Project Settings** > **Service accounts**, click **Generate new private key** to download your admin credentials.

Step 3: Environment Variables Configuration

Create a file named `.env.local` in the root directory and populate the required keys:

```
# Firebase Client Configuration (Browser)
NEXT_PUBLIC_FIREBASE_API_KEY="AIzaSy..."
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN="cursis-prod.firebaseio.com"
NEXT_PUBLIC_FIREBASE_PROJECT_ID="cursis-prod"
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET="cursis-prod.appspot.com"
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID="123456789"
NEXT_PUBLIC_FIREBASE_APP_ID="1:123456789:web:abcdef"

# Firebase Admin SDK Configuration (Serverless Routes)
FIREBASE_PROJECT_ID="cursis-prod"
FIREBASE_CLIENT_EMAIL="firebase-adminsdk-xxxxx@cursis-prod.iam.gserviceaccount.com"
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIIEvgIBADANBgk...=\n-----END PRIVATE KEY-----\n"

# Session Security & AI Secrets
AUTH_SESSION_SECRET="generate_a_random_32_character_secret_key"
```

```
GEMINI_API_KEY="AizaSy..."
```

Step 4: Next.js & Serverless Bundling Configuration

Ensure `next.config.ts` contains `serverExternalPackages: ['firebase-admin']` to prevent Turbopack native gRPC bundling errors on Vercel:

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  reactStrictMode: false,
  devIndicators: false,
  serverExternalPackages: ['firebase-admin'],
  experimental: {
    optimizePackageImports: ['lucide-react', 'framer-motion', 'three', '@react-three/fiber', '@react-three/drei'],
  },
};

export default nextConfig;
```


6. RUNNING LOCALLY & PRODUCTION DEPLOYMENT

Local Development & Production Verification

```
# Start local dev server (Hot reload enabled on http://localhost:3000)
npm run dev

# Run strict TypeScript and Next.js production build check
npm run build

# Launch local production server
npm run start
```

Deploying to Vercel (Edge Platform)

1. Push your latest changes to GitHub: `git push origin main`.
2. Go to <https://vercel.com/new> and import your `Cursis` repository.
3. In the **Environment Variables** section, paste all values from your `.env.local`.
4. Click **Deploy**. Vercel will compile all 47 routes and deploy the application globally.
5. In **Firebase Console > Authentication > Settings > Authorized domains**, add your Vercel deployment URL (e.g. `cursis.vercel.app`) to allow Google OAuth popups.

7. ENGINEERING BEST PRACTICES & ARCHITECTURE REFERENCE

Resilient Session Exchange

The `/api/auth/session` endpoint returns a self-contained, signed session token embedded in an `HttpOnly` cookie with `SameSite=Lax`. This eliminates reliance on blocking remote roundtrips during critical serverless invocations.

3D WebGL Performance

Three.js canvas contexts are wrapped with lazy loading and dynamic imports (`next/dynamic` with `ssr: false`) to ensure fast First Contentful Paint (FCP) and zero main-thread blocking.

CURSIS ENGINEERING ARCHITECTURE SPECIFICATION

Compiled & Verified • Production Release 2026 • Mukul / Cursis Core Team